

Distributional Reinforcement Learning with Function Approximation

Abstract

These notes give an overview of some theoretical foundations behind approximation based distributional reinforcement learning and newer methods using differentiable metrics on probability measures. We begin with linear value-function approximation in the standard scalar case, then we replace scalar values by return distributions and introduce the two classic representations, quantile and categorical, of probability distributions. Then, we go over how these representations can be used in function approximation using deep neural networks. Finally, we cover the recent research on distributional reinforcement learning using differentiable metrics on probability measures such as the MMD distances or Sinkhorn divergence.

Preliminaries

The first part of these notes follow the treatment of distributional reinforcement learning in [BDR23]. We first recall some of the preliminary definitions and results.

Definition 1: Finite Markov decision process and policy

A finite Markov decision process is given by

$$(\mathcal{X}, \mathcal{A}, \xi_0, P_X, P_R),$$

where $\mathcal{X}$ is a finite state space, $\mathcal{A}$ is a finite action space, ξ_0 is an initial-state distribution, $P_X(\cdot \mid x, a)$ is the next-state kernel, and $P_R(\cdot \mid x, a)$ is the reward distribution. A policy is a map

$$\pi : \mathcal{X} \rightarrow \mathcal{P}(\mathcal{A}), \quad x \mapsto \pi(\cdot \mid x),$$

and the discount factor satisfies $\gamma \in [0, 1)$. The markov chain induced by P_X and π gives rises to the trajectory measure P_π .

Given a policy π , the trajectory process is

$$\begin{aligned} X_0 &\sim \xi_0, & A_t &\sim \pi(\cdot \mid X_t), \\ R_t &\sim P_R(\cdot \mid X_t, A_t), & X_{t+1} &\sim P_X(\cdot \mid X_t, A_t), & t \geq 0. \end{aligned}$$

Starting from $X_0 = x$ and then following π , define the discounted return

$$G^\pi(x) = \sum_{t=0}^{\infty} \gamma^t R_t.$$

using this return distribution, one can naturally define the value function that tells us for each state, what the expected return is.

Definition 2: Value function and return distribution

The state-value function is the expected return

$$V^\pi(x) = \mathbb{E}_\pi[G^\pi(x)].$$

The return distribution at state x is

$$\eta_\pi(x) = \text{Law}(G^\pi(x)).$$

By definition, we can also write the value function as an integral using the return distribution as a measure.

$$V^\pi(x) = \int_{\mathbb{R}} z \eta_\pi(x)(dz).$$

Central to Reinforcement learning is the Bellman operator and its fixed point equation.

Definition 3: Bellman operator for scalar values

For a function $V : \mathcal{X} \rightarrow \mathbb{R}$, the Bellman operator for policy π is

$$(\mathcal{T}^\pi V)(x) = \mathbb{E}_{a \sim \pi(\cdot | x)} [\mathbb{E}[R + \gamma V(X') \mid X = x, a]].$$

Equivalently, the outer expectation samples the action from the policy and the inner expectation samples reward and next state from the MDP. The Bellman equation is

$$V^\pi = \mathcal{T}^\pi V^\pi.$$

There exists a stronger version of this Bellman equation, which is its distributional variant.

$$A \sim \pi(\cdot \mid x), \quad R \sim P_R(\cdot \mid x, A), \quad X' \sim P_X(\cdot \mid x, A),$$

the return satisfies

$$G^\pi(x) \stackrel{D}{=} R + \gamma G^\pi(X') \quad \text{given } X = x.$$

Taking laws gives

$$\eta_\pi(x) = \text{Law}(R + \gamma G^\pi(X') \mid X = x).$$

Definition 4: Distributional Bellman operator

For $r, z \in \mathbb{R}$, define what in [BDR23] is called the bootstrap map via

$$b_{r,\gamma}(z) = r + \gamma z.$$

If $\nu \in \mathcal{P}(\mathbb{R})$ and $Z \sim \nu$, then the pushforward $(b_{r,\gamma})_{\#}\nu$ is the law of $r + \gamma Z$:

$$(b_{r,\gamma})_{\#}\nu = \text{Law}(r + \gamma Z).$$

For a return-distribution function $\eta : \mathcal{X} \rightarrow \mathcal{P}(\mathbb{R})$, the distributional Bellman operator is

$$(\mathcal{T}^{\pi}\eta)(x) = \mathbb{E}_{a \sim \pi(\cdot|x)} [\mathbb{E} [(b_{R,\gamma})_{\#}\eta(X') \mid X = x, a]].$$

The return-distribution function η_{π} is its fixed point:

$$\eta_{\pi} = \mathcal{T}^{\pi}\eta_{\pi}.$$

Theorem 1: Wasserstein contraction of the distributional Bellman operator

For $p \in [1, \infty]$, let

$$\bar{w}_p(\eta, \nu) = \sup_{x \in \mathcal{X}} w_p(\eta(x), \nu(x)).$$

Then

$$\bar{w}_p(\mathcal{T}^{\pi}\eta, \mathcal{T}^{\pi}\nu) \leq \gamma \bar{w}_p(\eta, \nu).$$

Proof. We give the coupling proof from [BDR23, Proposition 4.15] for $p < \infty$. For each state y , take an optimal coupling $(Z(y), Z'(y))$ of $\eta(y)$ and $\nu(y)$. Couple the two Bellman updates by using the same sampled action, reward, and next state in both. Conditional on $X = x$, the resulting pair

$$\tilde{Z} = R + \gamma Z(X'), \quad \tilde{Z}' = R + \gamma Z'(X')$$

is a valid coupling of $(\mathcal{T}^{\pi}\eta)(x)$ and $(\mathcal{T}^{\pi}\nu)(x)$. Therefore

$$\begin{aligned} w_p^p((\mathcal{T}^{\pi}\eta)(x), (\mathcal{T}^{\pi}\nu)(x)) &\leq \mathbb{E}[|\tilde{Z} - \tilde{Z}'|^p \mid X = x] \\ &= \gamma^p \mathbb{E}[|Z(X') - Z'(X')|^p \mid X = x] \\ &\leq \gamma^p \sup_{y \in \mathcal{X}} w_p^p(\eta(y), \nu(y)). \end{aligned}$$

Taking the p th root and then the supremum over x proves the claim. The $p = \infty$ case uses the same coupling and the essential supremum. $\square$

Definition 5: Finite representations of return distributions

For a probability measure $\eta \in \mathcal{P}(\mathbb{R})$, let

$$F_\eta(t) = \eta((-\infty, t]), \quad F_\eta^{-1}(\tau) = \inf\{z \in \mathbb{R} : F_\eta(z) \geq \tau\}.$$

The two standard representations are:

$$\text{categorical:} \quad \eta(x) = \sum_{i=1}^m p_i(x) \delta_{\theta_i}, \quad p_i(x) \geq 0, \quad \sum_{i=1}^m p_i(x) = 1,$$

and

$$\text{quantile:} \quad \eta(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x)}, \quad \tau_i = \frac{2i-1}{2m}, \quad \theta_i(x) \approx F_{\eta(x)}^{-1}(\tau_i).$$

If $\mathcal{X}$ is large, storing a scalar value for every state is already expensive. In our case, we would need to store a full probability distribution for every state which is even more expensive. Hence, the rest of the notes which are based on [BDR23] will focus on value and action value function approximation, starting with the linear case.

1 Linear Function Approximation

We first discuss linear approximation for the state-value function

$$V^\pi : \mathcal{X} \rightarrow \mathbb{R}.$$

This section follows parts of Chapter 9 of [BDR23].

1.1 Function Approximation

In linear function approximation, we represent the value function as a linear combination of features. Similar to neural networks, the features here are allowed to be nonlinear.

Definition 6: State representation and linear value approximation

Let $n \in \mathbb{N}$. A state representation is a mapping

$$\phi : \mathcal{X} \rightarrow \mathbb{R}^n, \quad \phi(x) = \begin{pmatrix} \phi_1(x) \\ \vdots \\ \phi_n(x) \end{pmatrix}.$$

For $w \in \mathbb{R}^n$, the linear value function approximation is

$$V_w(x) = \phi(x)^\top w,$$

where each ϕ_i is a basis function and $\phi_i(x)$ is a feature.

The model is linear in its weights and

$$V_{\alpha w_1 + \beta w_2} = \alpha V_{w_1} + \beta V_{w_2}, \quad \nabla_w V_w(x) = \phi(x).$$

For finite $\mathcal{X}$, collect the row vectors $\phi(x)^\top$ in a feature matrix $\Phi \in \mathbb{R}^{|\mathcal{X}| \times n}$. Then

$$V_w = \Phi w, \quad \mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}.$$

The same can be done for action value function using $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^n$ and $Q_w(x, a) = \phi(x, a)^\top w$. Since we want to focus on what the distributional part changes about the approximation later, we will mostly discuss the state value case.

1.2 Optimal Linear Value Function Approximation

As states above, the equation

$$V_w = \Phi w$$

defines a linear system, which won't have a solution in general. Instead, a standard way to find the 'best' solution is to minimize the squared error. This can also be done here, however in [BDR23] it is mentioned that in RL we are often not equally interested in all states, hence we introduce a weighted least squares loss.

So, let $\xi \in \mathcal{P}(\mathcal{X})$ be the relative importance of the states and define

$$\|V - V'\|_{\xi,2} = \left(\sum_{x \in \mathcal{X}} \xi(x) (V(x) - V'(x))^2 \right)^{1/2}.$$

we are thus interested in the best approximation to V^π which solves

$$\min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_{\xi,2}.$$

and this is given by a weighted normal equation.

Theorem 2: Weighted normal equation

Let $\Xi = \text{diag}(\xi(x) : x \in \mathcal{X})$. If ξ has full support and the columns of Φ are linearly independent, the unique solution is

$$w^* = (\Phi^\top \Xi \Phi)^{-1} \Phi^\top \Xi V^\pi.$$

Thus the state weighting ξ is part of the approximation problem: changing how often states are sampled generally changes the best attainable weights.

The problem is, we can't iterate the Bellman operator since after applying it, we might not have a linear representation anymore. This motivates the following definition of a projected Bellman operator.

1.3 The Projected Bellman Operator

Definition 7: Projected Bellman operator

Define the weighted orthogonal projection $\Pi_{\phi,\xi}$ by

$$(\Pi_{\phi,\xi}V)(x) = \phi(x)^\top w^*, \quad w^* \in \arg \min_{w \in \mathbb{R}^n} \|V - V_w\|_{\xi,2}.$$

Approximate dynamic programming iterates

$$V_{k+1} = \Pi_{\phi,\xi} \mathcal{T}^\pi V_k.$$

Luckily, one can still show that this projected Bellman operator is a contraction and that the fixed point of the projected Bellman operator is close to the best approximation of the true value function. Hence, by iteration the Bellman equation like this, we will converge to something that is close to what we are trying to approximate!

Theorem 3: Projected Bellman contraction and error bound

Let ξ_π be a fully supported steady-state distribution under π . Then Π_{ϕ,ξ_π} is a nonexpansion in the ξ_π -weighted L_2 norm. Consequently,

$$\|\Pi_{\phi,\xi_\pi} \mathcal{T}^\pi V - \Pi_{\phi,\xi_\pi} \mathcal{T}^\pi V'\|_{\xi_\pi,2} \leq \gamma \|V - V'\|_{\xi_\pi,2}.$$

The projected operator has a unique fixed point

$$\hat{V}^\pi = \Pi_{\phi,\xi_\pi} \mathcal{T}^\pi \hat{V}^\pi,$$

Additionally, they show that it holds

$$\|\hat{V}^\pi - V^\pi\|_{\xi_\pi,2} \leq \frac{1}{\sqrt{1-\gamma^2}} \|\Pi_{\phi,\xi_\pi} V^\pi - V^\pi\|_{\xi_\pi,2}.$$

which means that the fixed point of the projected Bellman operator is close to the best approximation.

Proof. The proofs follows the argument from [BDR23, Theorem 9.8]. The transition operator P^π is nonexpansive and due to the orthogonality of the projection we have for every U ,

$$U = \Pi_{\phi,\xi_\pi} U + (I - \Pi_{\phi,\xi_\pi})U$$

is an orthogonal decomposition in the ξ_π -weighted inner product. Hence, by Pythagoras,

$$\|U\|_{\xi_\pi,2}^2 = \|\Pi_{\phi,\xi_\pi} U\|_{\xi_\pi,2}^2 + \|(I - \Pi_{\phi,\xi_\pi})U\|_{\xi_\pi,2}^2,$$

so $\|\Pi_{\phi,\xi_\pi} U\|_{\xi_\pi,2} \leq \|U\|_{\xi_\pi,2}$. Thus the composition with $\mathcal{T}^\pi$ is a γ -contraction. Banach's fixed-point theorem gives a unique fixed point and convergence of iterating the projected Bellman operator.

For the error estimate, we use orthogonality of the projection and $V^\pi = \mathcal{T}^\pi V^\pi$ which yields

$$\begin{aligned}\|\hat{V}^\pi - V^\pi\|_{\xi_\pi, 2}^2 &= \|\hat{V}^\pi - \Pi_{\phi, \xi_\pi} V^\pi\|_{\xi_\pi, 2}^2 + \|\Pi_{\phi, \xi_\pi} V^\pi - V^\pi\|_{\xi_\pi, 2}^2 \\ &\leq \gamma^2 \|\hat{V}^\pi - V^\pi\|_{\xi_\pi, 2}^2 + \|\Pi_{\phi, \xi_\pi} V^\pi - V^\pi\|_{\xi_\pi, 2}^2.\end{aligned}$$

Rearranging and taking square roots proves the bound. $\square$

However, while this result is nice, performing full Bellman update is expensive. A more practical tool is TD learning which can also be done in the approximate regime. Since we again want to focus on the distributional side, we don't mention it here and refer to [BDR23, Chapter 9.4] for more details.

2 Distributional Value Approximation

Distributional reinforcement learning replaces the scalar value function by a return-distribution function. In the following, we use the action-value version of the distributional Bellman operator defined in the preliminaries.

As highlighted in the prelims, the book mainly discusses two ways to parametrize the return distribution. In both cases we are talking about probability distribution, which do not form a linear space. This is a problem, since we cannot easily define an orthogonal projection. Hence, for now, we focus on the TD learning of the distributional case. First, we define the linear approximation for both cases.

Definition 8: Categorical return-distribution approximation

Fix the support of the atoms

$$S = \{\theta_1 < \dots < \theta_m\} \subset \mathbb{R}.$$

The categorical approximation keeps these locations fixed and learns their probabilities:

$$\eta_W(x, a) = \sum_{i=1}^m p_i(x, a; W) \delta_{\theta_i}.$$

to learn probabilities, one classically uses the softmax function of the approximated function. Hence, we have $\ell_i(x, a) = \phi(x, a)^\top w_i$, as our parametrization and

$$p_i(x, a; W) = \frac{\exp(\phi(x, a)^\top w_i)}{\sum_{j=1}^m \exp(\phi(x, a)^\top w_j)}.$$

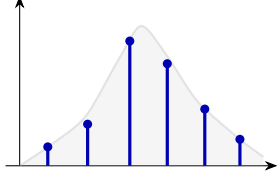
The quantile family instead parametrizes $\nu \in \mathcal{P}(\mathbb{R})$ with cumulative distribution function F_ν , through its quantile function by

$$F_\nu^{-1}(\tau) = \inf\{z \in \mathbb{R} : F_\nu(z) \geq \tau\}, \quad \tau \in (0, 1).$$

in the quantile representation, we try and approximate the positions of fixed quantile levels.

Categorical: fixed locations, learned masses

$$\eta(x) = \sum_{i=1}^m p_i(x) \delta_{\theta_i}$$



Quantile: equal masses, learned locations

$$\eta(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x)}$$

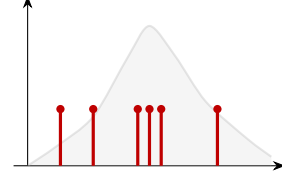


Figure 1: Comparison between categorical and quantile approximations.

Definition 9: Quantile return-distribution approximation

The general form of linear quantile return distribution approximation is

$$\theta_i(x, a) = \phi(x, a)^\top w_i, \quad w_i \in \mathbb{R}^d.$$

Equivalently, with $W = (w_1, \dots, w_N) \in \mathbb{R}^{d \times N}$,

$$\eta_W(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\phi(x, a)^\top w_i}.$$

In particular, we fix the probabilities to be equal, but allow the atom positions to change.

3 Distributional Approximate TD

The categorical TD learning is very similar to the scalar case, hence we focus on the quantile case and refer to [BDR23, Chapter 10].

For the quantile case, if we are given a policy π , sample a transition and next action

$$(x_t, a_t, r_t, x'_t, a'_t), \quad a'_t \sim \pi(\cdot \mid x'_t).$$

The distributional Bellman target is represented by the N target atoms

$$y_j = r_t + \gamma \theta_j(x'_t, a'_t), \quad j = 1, \dots, N.$$

The current prediction at (x_t, a_t) has atoms

$$z_i = \theta_i(x_t, a_t), \quad i = 1, \dots, N.$$

Definition 10: Quantile TD loss

Quantile regression uses the asymmetric absolute loss

$$\rho_\tau(u) = u(\tau - \mathbb{1}_{\{u < 0\}}).$$

The empirical Quantile TD loss is then

$$L(W) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}(y_j - z_i).$$

The residual $y_j - z_i$ compares an atom of the Bellman target distribution with the predicted τ_i -quantile at the current state and chosen action. Minimizing this loss moves $\eta_W(x_t, a_t)$ towards the bellman target

$$\frac{1}{N} \sum_{j=1}^N \delta_{r_t + \gamma \theta_j(x'_t, a'_t)} = (b_{r_t, \gamma})_{\#} \eta_W(x'_t, a'_t).$$

For linear quantiles, a subgradient (which we need since its not differentiable at 0) update is

$$w_{i,t+1} = w_{i,t} + \alpha_t \left(\frac{1}{N} \sum_{j=1}^N (\tau_i - \mathbb{1}_{\{y_j < z_i\}}) \right) \phi(x_t, a_t), \quad i = 1, \dots, N.$$

This is the exact analogue of the semi-gradient TD update:

$$\delta_t = r_t + \gamma Q_{w_t}(x'_t, a'_t) - Q_{w_t}(x_t, a_t)$$

If we are not given a policy, we can just greedily sample it at the next state.

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N \theta_j(x', a').$$

The target atoms become

$$y_j = r + \gamma \theta_j(x', a^*(x')), \quad j = 1, \dots, N.$$

4 Deep Distributional Reinforcement Learning

Now, instead of a linear feature representation, we use a neural network

$$\psi_\omega(x) \in \mathbb{R}^d,$$

where ω are the parameters and the final layers produce a distributional prediction in some sense, i.e. either quantiles or probabilities for the categorical case.

4.1 QR-DQN

QR-DQN is the deep version of Quantile TD. It uses a neural network to output N quantile locations for each action:

$$\theta_i(x, a; \omega), \quad i = 1, \dots, N, \quad a \in \mathcal{A}.$$

For each state-action pair, the predicted return distribution is

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a; \omega)}.$$

As in DQN, QR-DQN uses two sets of parameters. This is needed because the target network is the same as our current one, hence we fix the target parameters for a couple of iterations always. For a transition (x, a, r, x') , the greedy next action is selected by the mean of the target-network distribution:

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N \theta_j(x', a'; \bar{\omega}).$$

The target quantile atoms are

$$y_j = r + \gamma(1 - d)\theta_j(x', a^*(x'); \bar{\omega}), \quad j = 1, \dots, N.$$

The online prediction atoms for the sampled action are

$$z_i = \theta_i(x, a; \omega), \quad i = 1, \dots, N.$$

Definition 11: Quantile Huber loss

QR-DQN replaces the nonsmooth quantile loss ρ_τ by the quantile Huber loss; see [BDR23, Chapter 10]. Define

$$\mathcal{L}_\kappa(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq \kappa, \\ \kappa \left(|u| - \frac{1}{2}\kappa \right), & |u| > \kappa, \end{cases}$$

and

$$\rho_\tau^\kappa(u) = \left| \tau - \mathbb{1}_{\{u < 0\}} \right| \frac{\mathcal{L}_\kappa(u)}{\kappa}.$$

For small residuals the loss is quadratic and gives smoother gradients, in particular it is differentiable at 0. For large residuals it behaves linearly, preventing exploding gradients.

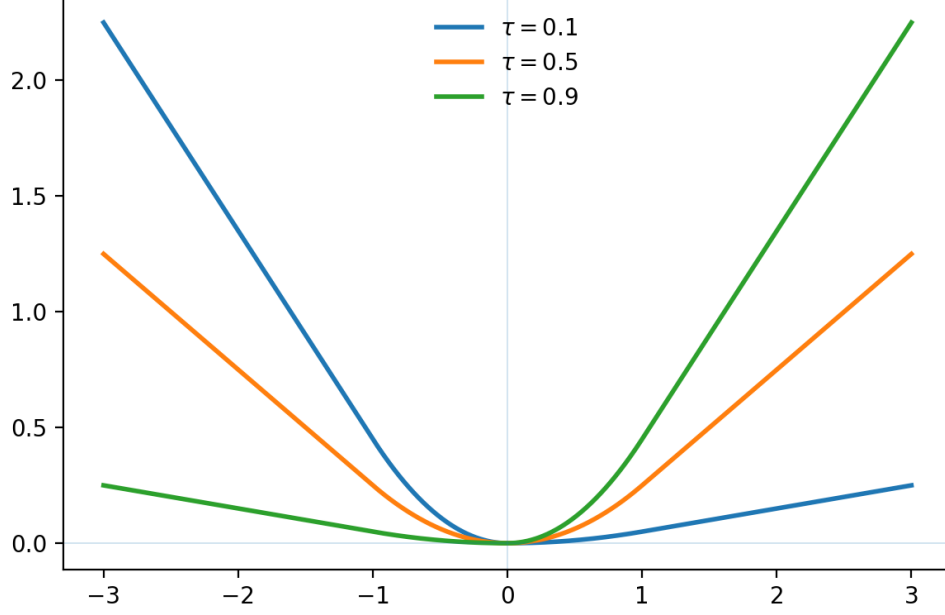


Figure 2: Quantile Huber losses for several quantile levels.

Definition 12: QR-DQN training objective

QR-DQN minimizes the pairwise quantile Huber loss

$$L_{\text{QR}}(\omega) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}^{\kappa}(y_j - z_i),$$

with quantile levels

$$\tau_i = \frac{2i - 1}{2N}.$$

If we have access to a minibatch $\mathcal{B}$ sampled from replay memory, the objective is just the sum over the above single transition loss.

For the categorical case, the most notable deep variant is C51 which essentially uses the same ideas that the linear approximation in both the scalar and distributional versions use, hence we will not discuss it here. For more details, see [BDR23, Chapter 10.2].

5 Particle Distributional DQN

Now we will discuss some recent work on Distributional DQN that does not utilize a fixed parametrization of the return distribution. Instead, a particle representation is used where we just take the empirical measure of multiple samples.

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{z_i(x, a; \omega)}$$

For such methods, the common template is:

1. choose a divergence or metric d between probability measures,
2. study whether the distributional Bellman operator contracts under the induced supremum metric which takes the maximum difference over the states and actions

$$\bar{d}(\eta, \nu) = \sup_{x, a} d(\eta(x, a), \nu(x, a)),$$

3. use the empirical version of d as a differentiable DQN loss.

The last point is important, since the standard Wasserstein distance is not differentiable and hence not that well suited for this problem.

5.1 MMD-DQN

This section follows [NTGV21]. The maximum mean discrepancy (MMD) is a metric on probability measures that is induced by a reproducing kernel Hilbert space. Depending on the choice of the kernel, many different metrics can be constructed. However not all kernels induce a metric.

Definition 13: Kernel mean embedding and maximum mean discrepancy

Let $k : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ be a continuous positive-definite kernel and let $\mathcal{H}_k$ be its reproducing kernel Hilbert space. For a probability measure μ , define the kernel mean embedding

$$\psi_\mu = \int k(z, \cdot) \mu(dz) \in \mathcal{H}_k.$$

The maximum mean discrepancy is

$$\text{MMD}_k(\mu, \nu) = \|\psi_\mu - \psi_\nu\|_{\mathcal{H}_k}.$$

Equivalently, if $Z, Z' \sim \mu$ and $Y, Y' \sim \nu$ are independent, then

$$\text{MMD}_k^2(\mu, \nu) = \mathbb{E}k(Z, Z') + \mathbb{E}k(Y, Y') - 2\mathbb{E}k(Z, Y).$$

If the kernel is characteristic, the embedding is injective, and $\text{MMD}_k(\mu, \nu) = 0$ implies $\mu = \nu$. With this, the MMD is really a metric.

Example 1: Characteristic kernels

Standard characteristic kernels on $\mathbb{R}^d$ include

$$\text{Gaussian: } k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right),$$

$$\text{Laplace: } k(x, y) = \exp\left(-\frac{\|x - y\|}{\sigma}\right),$$

$$\text{inverse quadratic: } k(x, y) = (c^2 + \|x - y\|^2)^{-\beta},$$

$$\text{unrectified distance: } k_\alpha(x, y) = -\|x - y\|^\alpha, \quad 0 < \alpha < 2,$$

where $\sigma, c, \beta > 0$. By contrast, the linear kernel $k(x, y) = x^\top y$ is not characteristic.

Nguyen et al. define the state-action supremum metric as hinted at in the introduction [NTGV21]

$$\text{MMD}_{\infty, k}(\eta, \nu) = \sup_{x, a} \text{MMD}_k(\eta(x, a), \nu(x, a)).$$

Sadly, the Bellman operator is not a contraction for arbitrary characteristic kernels, but we need some properties for them.

Theorem 4: MMD Bellman contraction, [NTGV21, Theorem 2]

Assume that $k = \sum_{i \in I} c_i k_i$ where $c_i \geq 0$ and every k_i has the following two properties. Firstly, it is translation invariant,

$$k(x + c, y + c) = k(x, y)$$

and somewhat scale invariant, i.e. for $b_{\gamma, 0}(z) = \gamma z$,

$$\text{MMD}_k((b_{\gamma, 0})_{\#} \mu, (b_{\gamma, 0})_{\#} \nu) = \gamma^{\alpha_i/2} \text{MMD}_k(\mu, \nu).$$

then, if we set $\alpha_* := \inf_{i \in I} \alpha_i$. The distributional Bellman operator satisfies

$$\text{MMD}_{\infty, k}(\mathcal{T}^\pi \eta, \mathcal{T}^\pi \nu) \leq \gamma^{\alpha_*/2} \text{MMD}_{\infty, k}(\eta, \nu).$$

In particular, the unrectified distance kernel k_α satisfies the assumptions. On the other hand, the Gaussian kernel does not satisfy the scale invariance since if we look at the scaled gaussian kernel we have

$$k_\sigma(\gamma x, \gamma y) = \exp\left(-\frac{\|\gamma x - \gamma y\|^2}{2\sigma^2}\right) = \exp\left(-\frac{\gamma^2 \|x - y\|^2}{2\sigma^2}\right) \neq \gamma^{\alpha/2} k_\sigma(x, y)$$

Theorem 5: Particle approximation rate [NTGV21, Theorem 3]

Let $P \in \mathcal{P}(\mathbb{R}^d)$ and choose n particles that minimize the MMD to P :

$$P_n = \frac{1}{n} \sum_{i=1}^n \delta_{x_i}, \quad (x_i)_{i=1}^n \in \arg \min_{(x_i)_{i=1}^n} \text{MMD}_k \left(\frac{1}{n} \sum_{i=1}^n \delta_{x_i}, P \right).$$

For every h in the unit ball of the corresponding RKHS,

$$\left| \int h dP_n - \int h dP \right| = O(n^{-1/2}).$$

This rate is independent of the ambient dimension; the paper contrasts it with the usual $O(n^{-1/d})$ empirical 1-Wasserstein rate when $d > 2$.

With the contraction result in hand we can now straight forwardly define the particle DQN method. For a transition (x, a, r, x') and target-network parameters $\bar{\omega}$, the particle DQN target is constructed by first choosing the greedy next action

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N z_j(x', a'; \bar{\omega})$$

and then shifting the target particles:

$$y_j = r + \gamma z_j(x', a^*(x'); \bar{\omega}), \quad j = 1, \dots, N.$$

With current particles $z_i = z_i(x, a; \omega)$, the empirical MMD loss is evaluated as

$$L_{\text{MMD}}(\omega) = \frac{1}{N^2} \sum_{i, i'=1}^N k(z_i, z_{i'}) + \frac{1}{N^2} \sum_{j, j'=1}^N k(y_j, y_{j'}) - \frac{2}{N^2} \sum_{i, j=1}^N k(z_i, y_j).$$

and this can be differentiated through autograd with respect to the parameters of our networks that produced the samples.

Suprisingly and disagreeing with the theory, we have better empirical performance when we choose the gaussian kernel which means that a distance performs well under which the Bellman Operator is not a contraction.

5.2 SinkhornDRL

As already stated, the Wasserstein distance is not a differentiable metric. This paper [Sun+24] uses a regularized version which makes it work.

Definition 14: Regularized optimal transport and Sinkhorn divergence

For a cost function $c : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_+$, the standard OT problem is the following minimization

$$\text{OT}_c(\mu, \nu) = \inf_{\pi \in \Pi(\mu, \nu)} \int c(z, y) \pi(dz, dy),$$

where $\Pi(\mu, \nu)$ is the set of couplings with marginals μ and ν . The entropic version is

$$\text{OT}_c^\varepsilon(\mu, \nu) = \inf_{\pi \in \Pi(\mu, \nu)} \left\{ \int c(z, y) \pi(dz, dy) + \varepsilon \text{KL}(\pi \mid \mu \otimes \nu) \right\}.$$

However, this version is not a metric anymore since for two measures that are the same, the regularized Wasserstein might be non zero. This is why the paper uses what they call the debiased Sinkhorn divergence

$$\bar{W}_{c, \varepsilon}(\mu, \nu) = \text{OT}_c^\varepsilon(\mu, \nu) - \frac{1}{2} \text{OT}_c^\varepsilon(\mu, \mu) - \frac{1}{2} \text{OT}_c^\varepsilon(\nu, \nu).$$

While this solves the issue of it being non zero when both entries are the same, this still does not satisfy the triangle inequality.

Based on this $\bar{W}_{c, \varepsilon}$, the paper defines the supremum metric as before

$$\bar{W}_{c, \varepsilon}^\infty(\eta, \nu) = \sup_{x, a} \bar{W}_{c, \varepsilon}(\eta(x, a), \nu(x, a)).$$

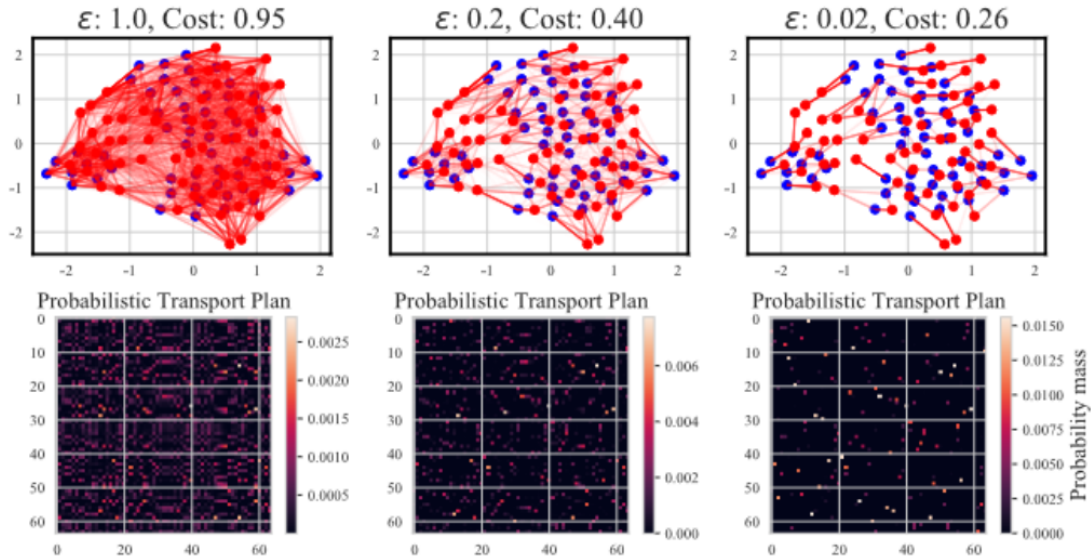


Figure 3: Regularization controls how much 'spread out' the mass is, since we bias the optimization towards the product measure which is 'the most spread out'.

For $c(x, y) = \|x - y\|^\alpha$, define the supremum divergence

$$\bar{W}_{c, \varepsilon}^\infty(\eta, \nu) = \sup_{x, a} \bar{W}_{c, \varepsilon}(\eta(x, a), \nu(x, a)).$$

Theorem 6: Reg. Wasserstein Bellman contraction, [Sun+24, Theorem 4.1]

Define

$$\bar{\Delta}_\varepsilon(\gamma, \alpha) = \gamma^\alpha \left(1 - \sup_{\mu, \nu} \lambda_\varepsilon(\mu, \nu) \right) + \sup_{\mu, \nu} \lambda_\varepsilon(\mu, \nu),$$

where

$$\lambda_\varepsilon(\mu, \nu) = \frac{\varepsilon \text{KL}(\pi^* \mid \mu \otimes \nu)}{\bar{W}_{c, \varepsilon}(\mu, \nu)} \in (0, 1)$$

and π^* is the optimal regularized coupling. This implies

$$\bar{W}_{c, \varepsilon}^\infty(\mathcal{T}^\pi \eta, \mathcal{T}^\pi \nu) \leq \bar{\Delta}_\varepsilon(\gamma, \alpha) \bar{W}_{c, \varepsilon}^\infty(\eta, \nu).$$

The rate lies between the unregularized Wasserstein rate also discussed in the book and the regularization dominated rate:

$$\bar{\Delta}_\varepsilon(\gamma, \alpha) \in (\gamma^\alpha, 1).$$

The particle learning rule has the same Bellman target construction as MMD-DQN. With current empirical measure

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{z_i(x, a; \omega)}$$

and target empirical measure

$$\hat{\eta}_{\bar{\omega}}(x, a, r, x') = \frac{1}{N} \sum_{j=1}^N \delta_{r + \gamma z_j(x', a^*(x'); \bar{\omega})},$$

the loss is

$$L_{\text{RW}}(\omega) = \bar{W}_{c, \varepsilon}(\eta_\omega(x, a), \hat{\eta}_{\bar{\omega}}(x, a, r, x')).$$

The Sinkhorn algorithm gives a differentiable approximation to this quantity for empirical measures, which makes it usable as a neural-network loss.

Theorem 7: Wasserstein–MMD interpolation [Sun+24, Theorem 1]

The regularization parameter interpolates between two geometries:

$$\varepsilon \rightarrow 0 \quad \Rightarrow \quad \bar{W}_{c, \varepsilon}(\mu, \nu) \rightarrow 2W_c^\alpha(\mu, \nu),$$

whereas

$$\varepsilon \rightarrow +\infty \quad \Rightarrow \quad \bar{W}_{c, \varepsilon}(\mu, \nu) \rightarrow \text{MMD}_{k_\alpha}^2(\mu, \nu), \quad k_\alpha = -c.$$

Thus the small-regularization endpoint recovers transport geometry, while the large-regularization endpoint recovers the MMD induced by the negative cost kernel.

This is why the presentation treats SinkhornDRL as a bridge between Wasserstein-style distributional RL and MMD-DQN: it keeps a transport-based objective while making the empirical particle loss smooth enough for deep learning.

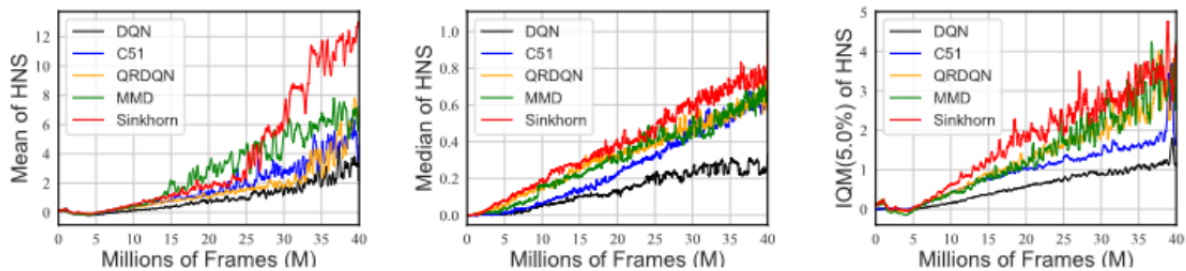


Figure 4: Atari aggregate performance reported in the presentation. The three panels show the mean, median, and interquartile mean of human-normalized scores over training.

References

- [BDR23] Marc G. Bellemare, Will Dabney, and Mark Rowland. *Distributional Reinforcement Learning*. MIT Press, 2023.
- [NTGV21] Thanh Nguyen-Tang, Sunil Gupta, and Svetha Venkatesh. “Distributional Reinforcement Learning via Moment Matching”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 10. 2021, pp. 9144–9152. DOI: 10.1609/aaai.v35i10.17104.
- [Sun+24] Ke Sun et al. “Distributional Reinforcement Learning with Regularized Wasserstein Loss”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024.